

Christopher Young
Nathan Howell
Logan Douglas

SLO 8 - Design Experience

All video references are attached with this report. Any non-video references can be found in the Index. The Index provides the directory locations for any other media (images or draw.io files) in our repository.

Overall Final Design

The final project our group developed was a stock screening application designed to predict the price of a stock a few hours into the future. In addition to price prediction, the application provides information about trading stocks to help users better understand the market. The goals we had were to create a tool that gives new users a clear starting point, creating a stock screener that can be used to filter stocks down based on criteria that a user desires, a prediction functionality that tries to predict a stocks price within a short period of time, develop a feature to show how much a person would have made if they invested a number of shares on a given day, and create an intuitive UI to tie all of this functionality together.

Incrementum has become the culmination of our design experience as it incorporates everything we have done while getting our degree. Aspects of Cloud and Mobile development, Web Development, Distributed Development, Database, Survey of Languages, and many small things built up since we started the software journey.

During cloud development we learned about hosting services on Azure and other cloud services. In our project we applied this by hosting our database on the cloud as other options weren't fitting the needs we had for the project. We contemplated hosting it ourselves or on the class dedicated server but this didn't get us the results we were looking for.

A large portion of our final software project borrowed our knowledge from Web Development. We had to use knowledge from Backend and Advanced Frontend. Our knowledge for how to create and work with APIs came as a result from what we gained moving forward from the backend (Figure 1). Creating our intuitive UI was the result of knowledge we gained from the Advanced Frontend class giving us ideas about what color conveys, providing structural hierarchy, and giving us experience with what having an intuitive UI really means. (See AppTour.mp4)

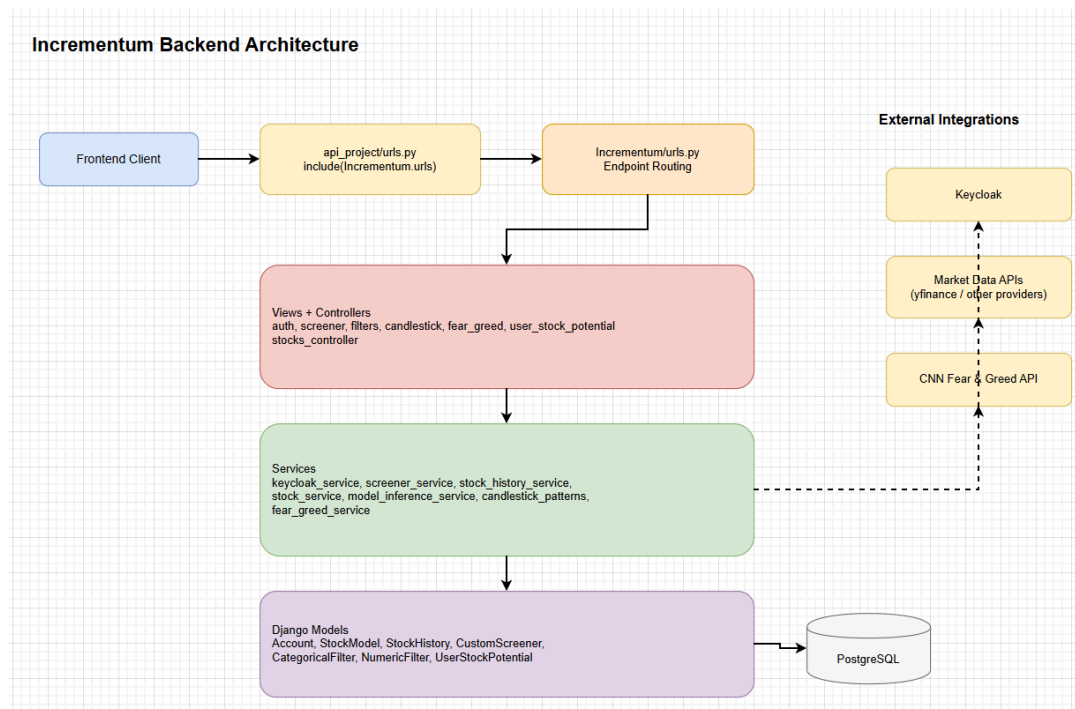


Figure 1. Backend Architecture (See Backend-Architecture.drawio)

Distributed development lets us know in order to handle large amounts of data for a large number of users we would have to scale services to be able to handle it. We used this knowledge in our project when creating data fetchers that we could scale up or down based on usage, user count, or other unexpected factors (Figure 2). This gave us an application that we could scale however much we wanted.

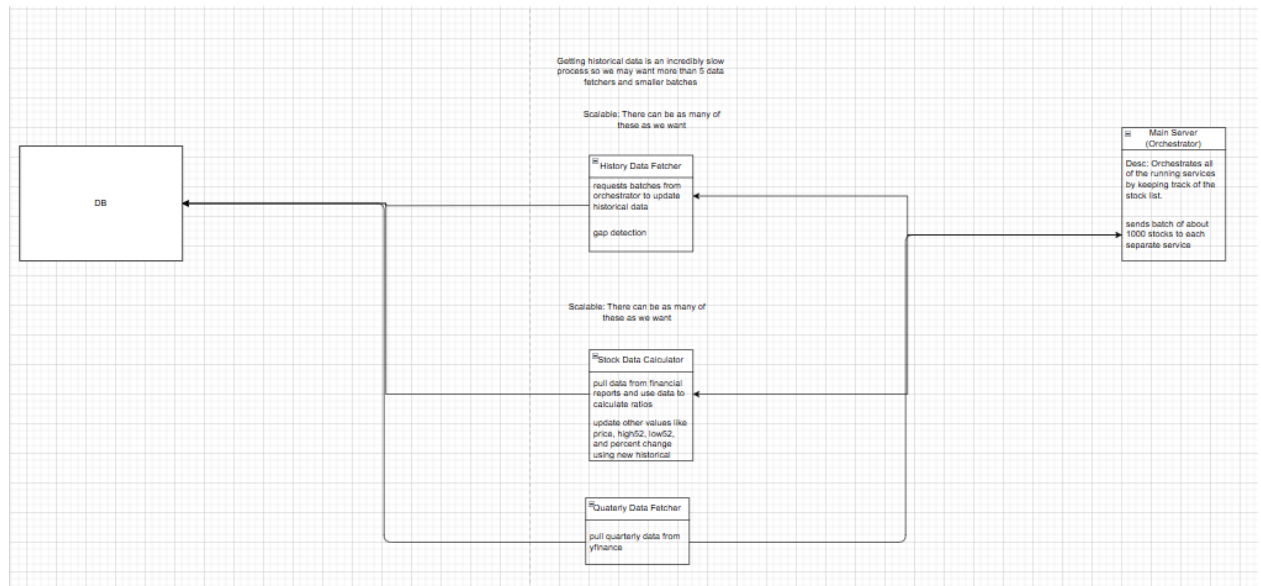


Figure 2. Data Fetchers Flow Chart (See DataFetchersAndCalculator.png)

The course about Databases gave us the knowledge and practice for creating the relational database we used for the large amounts of data we had to handle in this project. Specifically the class taught us about creating associations between tables such as storing a user id with the screener table to link these two together (Figure 3).

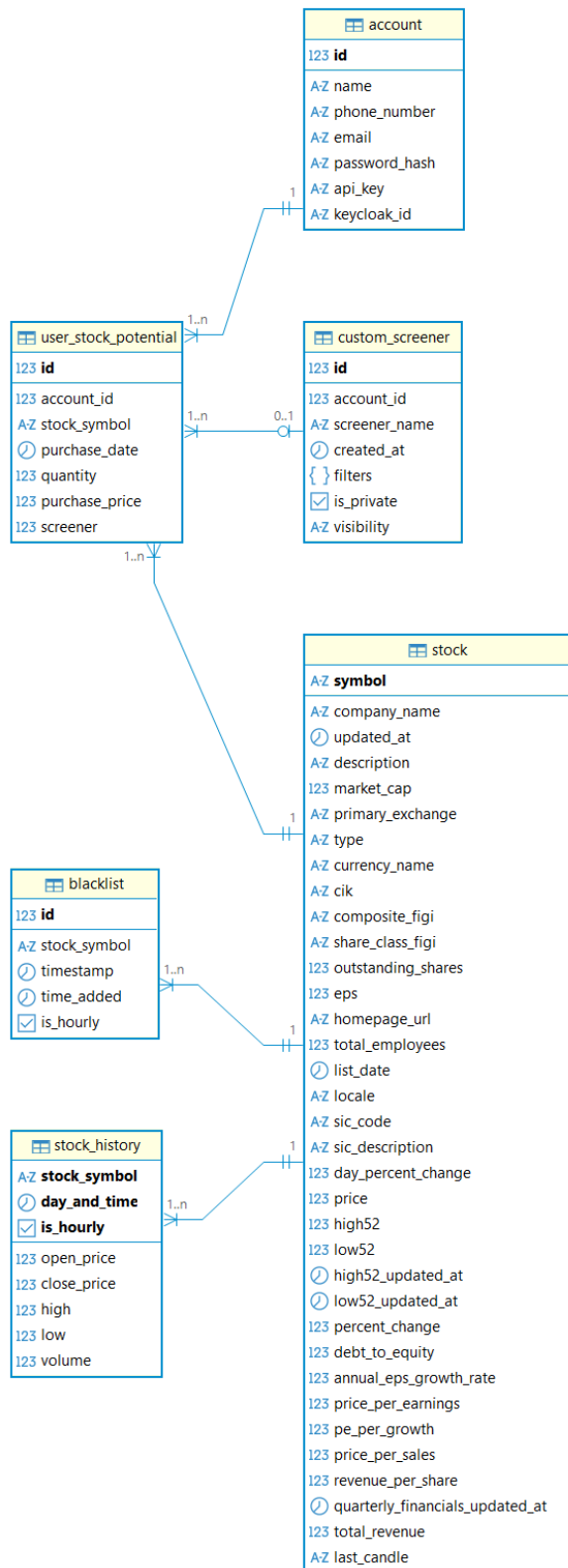


Figure 3. Database Schema (See DataBaseSchema.png)

For this project we decided to work with Python rather than the C# we had been taught throughout the year. We all had some experience but we weren't as familiar as we could have been. The knowledge of research and practicing with other languages gained from Survey of Languages gave us the confidence to use Python without worrying about how much time it would take us to gain a strong understanding of the language. We employed different practicing strategies that may have slowed us down at the start of this project but gave us the privilege of moving to Python for this project. (See Api.mp4)

The culmination of this experience allowed us to create this project by providing a foundation that we could then expand upon during the project. The coursework we have done gave us the ability to move forward with confidence in making decisions, and adapting to challenges presented like learning a new language or working with scalable systems. This experience allowed us to develop a much more polished application. Without these individual experiences, the project would have taken significantly longer to complete with a lower degree of quality, or may not have been possible at all.

Engineering Standards and Practices

The most important standards for us were clean and maintainable code, ethics, and good user experiences. For each meeting, everyone would have the chance to share the results of their assigned story and each team member would have a chance to review the code in the PR. Code reviews would often reveal mistakes in the code that we wouldn't want to be merged into the main branch (See CommitHistory.mp4 for

commit strategy). We made sure to track this progress on our kanban board (See KanbanBoard.mp4).

This would ensure that the code was clean and that it was doing what it was supposed to. We also had strict linting steps in our CI/CD pipeline (see CiCd.mp4) to keep the code clean and an automated testing step to ensure that the code followed the expected behavior. We also had descriptive logs with a grafana dashboard so if any errors did get past us, we would be able to easily debug them using the logs.

Section 1 of IEEE states that we're "to uphold the highest standards of integrity, responsible behavior, and ethical conduct in professional activities." We often went to our instructor for counseling, but in order to receive good feedback, we needed to be honest with the state of our team and the app. This semester, we did more frequent demos with him and it would give him a good idea of where we were at.

He could see from the demos how well we were working together and we would give him a report on how our team was doing. There were some weeks when we didn't work as effectively, but we were able to tell him and he would give us guidance for ways that we could use our time more effectively. By being honest with him, he was able to help motivate us and give us good feedback on how our app could be improved.

This was important for this project because our project relied heavily on good and accurate data. A small bug could mess up one of our data columns and so we needed to be honest when we did make a mistake. Otherwise, we would have ignored the bug and gave our users false information. For example, we needed to be careful with the percent change calculation because the sign could accidentally be flipped from a negative to a positive change or vice versa.

Section 2 says “To treat all persons fairly and with respect, to not engage in harassment or discrimination, and to avoid injuring others.” We had spent a lot of time together and would have a lot of arguments. While there were arguments between team members, we were all respectful of each other's opinions and ideas. Everyone was given a chance to voice their ideas and we were all able to contribute to the design of our app.

This project was Logan's idea that we approved of, but this still meant that we needed to be fair to everyone and remove any bias that could affect the design of our app. Sometimes we had flawed ideas and we needed to argue to reveal those flaws. This leads us to good user experiences. We would come up with a lot of design ideas and would demo them using figma charts.

We often made mistakes with our design that would only be revealed with a 3rd party's feedback. We would go to our instructor for guidance on how to improve our app. He would point out areas of confusion and would give us ideas for removing them. While the instructor was a good source of feedback, we needed more feedback from our potential users.

We did multiple rounds of user testing to receive this feedback. This semester, we each had the opportunity to do some user testing with both people that we knew and strangers. It was better to test with people that we didn't know though, because there was less bias. These users would reveal points of confusion by trying to use our app. There was a lot of confusion, but we managed to fix a lot of these points using the feedback that we had received.

If it wasn't for these user tests, our app would have been much more confusing to use. We would know how to use it because we are the ones that made it, but that wouldn't mean anything to anyone else. We aren't the only ones that would be using our app, and so having an intuitive UI was important.

Constraints

Database Performance

One of the constraints that we faced was with the database performance. We were constantly making calls to the database either by pushing or pulling stock info. It was to such a degree that when we were using a kubernetes database the queries were extremely slow.

Our final solution to this was to begin paying for (with school credits so it was essentially free anyways) a database on Microsoft Azure. This opened up way more processing space for us to actually do all the operations that we needed to do. It is one of the solutions that we would use if we were to make this public. The design that we used for our project (namely storing all of our datapoints in a database) meant that we could swap that database out at will with no real issue.

Database Size

This issue is similar but slightly different. We had over a hundred and fifty million rows in some of the tables in our database (See Row Count) that we would need to filter down to 1 for some queries. One of the calculation queries (See Problem Query) that

we were trying to do was trying to group all stock_history rows for each of the stocks and find the max value for each of them. This is a very intensive query. We wanted the app to be as responsive as possible and a query with several of these calculations was taking upwards of 10 minutes.

Our solution was to use the antipattern of making these calculations whenever they change and storing them in the database. Our design included several servers that essentially just calculate and push the calculated data to the database. Admittedly this is an anti-pattern but it is one that we are intentionally breaking for performance's sake. We make up for this in several ways such as refreshing this data at appropriate times.

API Limitations

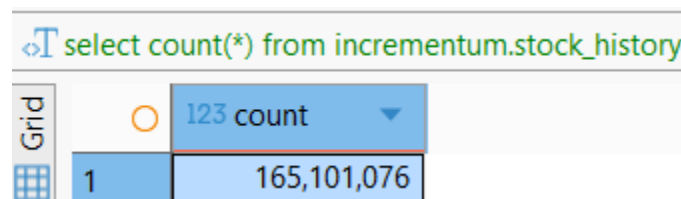
The constraint that we had with this one was definitely one of data. Our desire was to perform calculations using stock data (See StockDataFetcher.mp4). This uses tons and tons of data. We needed some sort of way to get this data. Originally we decided to use yahoo finance because it was free (sort of a budgeting constraint). We used yfinance, a python library that scrapes the yahoo stocks info pages. Problems with this library/api included: rate limiting, missing stocks, and missing datapoints. It was useful as a proof of concept but wasn't the route that we should have been taking.

We decided afterwards that it was better for moral and practical reasons to switch to a different api namely Polygon. The practical benefits of switching fixed most of these problems although we still struggled with missing data points which is a problem of funding more than anything. After we made this switch we had to remove references to yfinance from our code. We then added at this point better seams to accommodate

switching out apis that we used and even removed the responsibility of calling this api from our own backend api.

Index

- Row Count: This is the exact number of rows that our stock history table has.



The screenshot shows a SQL query editor with the query: `select count(*) from incrementum.stock_history`. Below the query, a table displays the result. The table has two columns: an index column and a 'count' column. The first row shows the index '1' and the count '165,101,076'.

	count
1	165,101,076

- Problem Query: The biggest problem with it is we are trying to get a calculated value for each stock, meaning without optimization we are searching through every single row and that takes time.

```
SELECT
  s.*,
  sh_high.high as week_52_high,
  sh_low.low as week_52_low
FROM incrementum.stock s
LEFT JOIN (
  SELECT stock_symbol, MAX(high) as high
  FROM incrementum.stock_history
  WHERE day_and_time >= NOW() - INTERVAL '1 year'
  GROUP BY stock_symbol
) sh_high ON sh_high.stock_symbol = s.symbol
LEFT JOIN (
  SELECT stock_symbol, MIN(low) as low
  FROM incrementum.stock_history
  WHERE day_and_time >= NOW() - INTERVAL '1 year'
  GROUP BY stock_symbol
) sh_low ON sh_low.stock_symbol = s.symbol
```

Design Folders

Each section of this represents a directory in our repository. The files under each section are in that directory.

Designs/

- Account Page.drawio.png: Account page design diagram for how the User will use it
- Login Page.drawio.png: Login page design diagram for how Users can log in
- Screener page Design.drawio (1).png: Screener page design diagram for how users interact with the screener
- Screening Design.drawio.png: New design for stock screening that doesn't rely on yfinance endpoints
- Stock Data Fetch Design v2.drawio.png: Another version of our old stock data fetchers design diagram
- Stock History Data Fetching.drawio.png: Includes original and final designs for the stock data fetchers/calculators

Designs/Old/

- Custom Collections.drawio.png: Requests diagram showing the flow of using our collections feature. This is back when we had collections
- GetUpdatedStockList.drawio.png: This is when we were using Finnhub to get the list of stock symbols
- NumericScreener.pu: Old screening design when we used the yfinance screen endpoint

- Prediction_Trend Data Design.drawio.png: This was our original plan for trend analysis models
- Stock Data Fetching.drawio.png: This is a data flow diagram for when we were using yfinance for everything
- StockGetterFlow.drawio.png: Old design for when we were using yfinance to get stocks
- Trends Design.drawio.png: Another design for trend analysis data
- Watchlist and Account DB.jpg: Old database schemas back when we had the watchlist feature
- Watchlist Design.drawio.png: Request flow diagram for the old watchlist feature

SLO/SLO8/

- Backend-Architecture.drawio: Architecture diagram for our current Incrementum api
- DataBaseSchema.png: Current database schema with all of the columns
- DataFetchersAndCalculator.png: The final design for our data fetching and calculating
- DataFetchersOriginal: The original design for our app
- HelpPage.png: Screenshot of our Help page
- PotentialGains.png: Screenshot of potential gains
- ScreenerPage.png: Screenshot of screener page
- SearchResults.png: Screenshot of search results page

- Stock History Data Fetching.drawio: Includes very first design and the final design of our stock data fetchers/calculators